

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN



Data Mining

Lab 02: Frequent Itemset Mining - FP-Growth*

Group 15

Sinh viên thực hiện:

Nguyễn Văn Linh (23122003)

Trần Chấn Hiệp (23122026)

Nguyễn Thị Mỹ Kim (23122040)

Giảng viên hướng dẫn:

Th.S Nguyễn Thị Thu Hằng

Th.S Nguyễn Ngọc Đức

Th.S Lê Nhựt Nam

Contents

1	Nền tảng lý thuyết	4
1.1	Bài toán Frequent Itemset Mining	4
1.1.1	Cơ sở dữ liệu giao dịch	4
1.1.2	Độ hỗ trợ (Support)	4
1.1.3	Tập phổ biến, tập đóng, tập tối đại	4
1.1.4	Tính chất Apriori (Downward Closure)	5
1.2	Thuật toán FP-growth*	5
1.2.1	Ý tưởng cốt lõi	5
1.2.2	Các định nghĩa cần thiết cho FP-growth*	6
1.2.3	Danh mục ký hiệu và chú thích các biến trong thuật toán	10
1.2.4	Mã giả thuật toán - Pseudocode	11
1.2.5	Phân tích độ phức tạp	13
1.2.6	So sánh lịch sử	15
2	Ví dụ minh họa tay (Step-by-step)	17
2.1	Ví dụ 1: cơ sở	17
2.1.1	Cơ sở dữ liệu	17
2.1.2	Xây dựng FP-tree $T_\emptyset$, FP-array $A_\emptyset$	17
2.1.3	Quá trình khai thác tập phổ biến bằng FP-growth* theo thuật toán 1	21
2.2	Ví dụ 2: Tình huống đặc biệt	24
2.2.1	Mô tả tình huống đặc biệt	24
2.2.2	Cơ sở dữ liệu	24
2.2.3	Xây dựng FP-tree	25
2.2.4	Xử lý trong FPgrowth*	25
2.2.5	Ý nghĩa kỹ thuật của trường hợp tối ưu này	25
3	Cài đặt	27
3.1	Môi trường và công cụ	27
3.2	Kiến trúc mã nguồn	28
3.3	Yêu cầu cài đặt	28
3.3.1	Cài đặt cơ bản	28
3.3.2	Tối ưu hóa bộ nhớ và tốc độ	29

3.3.3	Xử lý đầu vào/đầu ra	30
4	Thực nghiệm và đánh giá	31
4.1	Tập dữ liệu benchmark	31
4.2	Kết quả thực nghiệm	31
4.2.1	Kiểm tra tính đúng đắn (Correctness)	31
4.2.2	Thời gian chạy theo <i>minsup</i>	32
4.2.3	Số lượng FI theo <i>minsup</i>	33
4.2.4	Bộ nhớ sử dụng	33
4.2.5	Khả năng mở rộng (Scalability)	34
4.2.6	Ảnh hưởng của độ dài giao dịch trung bình	35
4.3	Phân tích kết quả	35
4.3.1	Điểm mạnh	35
4.3.2	Điểm yếu	36
4.3.3	Đề xuất tối ưu hóa tiếp theo	36
5	Ứng dụng thực tế	37
5.1	Phân tích giỏ hàng (Market Basket Analysis)	37
5.1.1	Dữ liệu	37
5.1.2	Phương pháp	37
5.1.3	Kết quả	37
5.1.4	Ý nghĩa kinh doanh	38

Bảng phân công

Tên	Nhiệm vụ	Mức độ hoàn thành
Nguyễn Thị Mỹ Kim	Cài đặt thuật toán + Report	100%
Nguyễn Văn Linh	Phân tích thuật toán + Report	100%
Trần Chấn Hiệp	Chạy benchmark, Ứng dụng + Report	100%

Table 1: Bảng phân công nhiệm vụ thực hiện Lab 1

Chapter 1

Nền tảng lý thuyết

1.1 Bài toán Frequent Itemset Mining

1.1.1 Cơ sở dữ liệu giao dịch

Định nghĩa 1.1 (Cơ sở dữ liệu giao dịch). Cho $\mathcal{I} = \{i_1, i_2, \dots, i_m\}$ là tập tất cả các item. Một **cơ sở dữ liệu giao dịch** (transaction database) là tập hữu hạn $\mathcal{D} = \{T_1, T_2, \dots, T_n\}$ trong đó mỗi **giao dịch** $T_k \subseteq \mathcal{I}$.

Định nghĩa 1.2 (Tập item – Itemset). Một **itemset** X là bất kỳ tập con không rỗng của $\mathcal{I}$, tức $X \subseteq \mathcal{I}$. Kích thước của X là $|X|$; X được gọi là k -itemset nếu $|X| = k$. Ta nói giao dịch T_k *chứa* X nếu $X \subseteq T_k$.

1.1.2 Độ hỗ trợ (Support)

Định nghĩa 1.3 (Support). Với itemset $X \subseteq \mathcal{I}$:

- **Support tuyệt đối - Support count**: $\text{sup}_{\text{abs}}(X) = |\{T \in \mathcal{D} : X \subseteq T\}|$
- **Support tương đối**: $\text{sup}(X) = \frac{\text{sup}_{\text{abs}}(X)}{|\mathcal{D}|}$

1.1.3 Tập phổ biến, tập đóng, tập tối đại

Định nghĩa 1.4 (Tập phổ biến – Frequent Itemset). Cho ngưỡng $\text{minsup} \in (0, 1]$ (tương đối) hoặc $\text{minsup} \in \mathbb{Z}^+$ (tuyệt đối). Itemset X là **tập phổ biến** (frequent itemset – FI) nếu $\text{sup}(X) \geq \text{minsup}$.

Định nghĩa 1.5 (Tập đóng – Closed Frequent Itemset). Itemset phổ biến X là **tập đóng** (CFI) nếu không tồn tại $Y \supsetneq X$ sao cho $\text{sup}(Y) = \text{sup}(X)$.

Định nghĩa 1.6 (Tập tối đại – Maximal Frequent Itemset). Itemset phổ biến X là **tập tối đại** (MFI) nếu không tồn tại itemset phổ biến Y nào sao cho $X \subsetneq Y$.

Tính chất 1.1 (Quan hệ giữa FI, CFI, MFI).

$$MFI \subseteq CFI \subseteq FI$$

Tập MFI nhỏ gọn nhất nhưng mất thông tin support; CFI đủ để tái tạo toàn bộ FI với support chính xác.

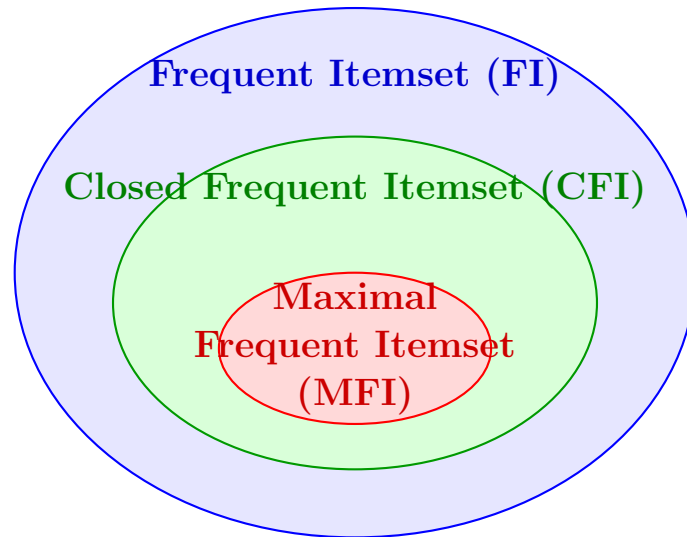


Figure 1.1: Biểu đồ Venn thể hiện mối quan hệ: $MFI \subseteq CFI \subseteq FI$

1.1.4 Tính chất Apriori (Downward Closure)

Tính chất 1.2 (Anti-monotone của Support). Với mọi $X, Y \subseteq \mathcal{I}$: nếu $X \subseteq Y$ thì $\text{sup}(X) \geq \text{sup}(Y)$.

Proof. Nếu giao dịch T chứa Y thì T cũng chứa X (vì $X \subseteq Y$). Do đó $\{T : Y \subseteq T\} \subseteq \{T : X \subseteq T\}$, suy ra $\text{sup}(Y) \leq \text{sup}(X)$. $\square$

Tính chất 1.3 (Tính chất Apriori). Nếu X không phổ biến thì mọi tập cha $Y \supsetneq X$ cũng không phổ biến. Nếu X phổ biến thì mọi tập con $Y \subsetneq X$ cũng không phổ biến.

Proof. Tính chất được suy ra trực tiếp từ **Tính chất 1.2**. $\square$

Tính chất này là nền tảng để *tỉa không gian tìm kiếm* trong tất cả các thuật toán FIM.

1.2 Thuật toán FP-growth*

1.2.1 Ý tưởng cốt lõi

Thuật toán **FP-growth*** (Grahne & Zhu, 2005 [1]) là phiên bản cải tiến của FP-growth (Han et al., 2000 [2]) với kỹ thuật **FP-array** nhằm loại bỏ một trong hai lần duyệt FP-tree trong mỗi lần gọi đệ quy.

Vấn đề của FP-growth gốc: Với mỗi item i trong cây T_X , cần *hai lần duyệt* danh sách liên kết (node-link) để xây dựng conditional FP-tree $T_{X \cup \{i\}}$:

1. Lần duyệt 1: đếm support của các item trong conditional pattern base.
2. Lần duyệt 2: thực sự xây dựng cây mới $T_{X \cup \{i\}}$.

Thực nghiệm cho thấy *80% CPU time* của FP-growth dùng cho việc duyệt FP-tree.

Giải pháp FP-array: Xây dựng đồng thời với cây T_X một ma trận A_X (FP-array) lưu số lần đồng xuất hiện của mọi cặp item. Khi đó, thông tin từ Lần duyệt 1 được đọc trực tiếp từ A_X trong $O(m)$ thay vì $O(N \times \text{depth})$.

Kết quả: FP-growth* nhanh hơn *gần gấp đôi* FP-growth trên dữ liệu thưa (sparse) và là một trong những thuật toán nhanh nhất.

1.2.2 Các định nghĩa cần thiết cho FP-growth*

Trước tiên ta sẽ tìm hiểu về FP-tree và cách khai thác tập phổ biến. Sau đó, ta sẽ tìm hiểu kĩ thuật FP-array giúp ta giảm thời gian duyệt cây.

FP-tree

Một **FP-tree** T là cây tiền tố (prefix tree) được xây dựng từ $\mathcal{D}$ sau khi lọc và sắp xếp item theo thứ tự giảm dần support. Mỗi nút chứa ba trường: **item-name**, **count**, **node-link**. Kèm theo T là **header table** lưu danh sách item tần suất theo thứ tự giảm dần và con trỏ đến node-link chain tương ứng.

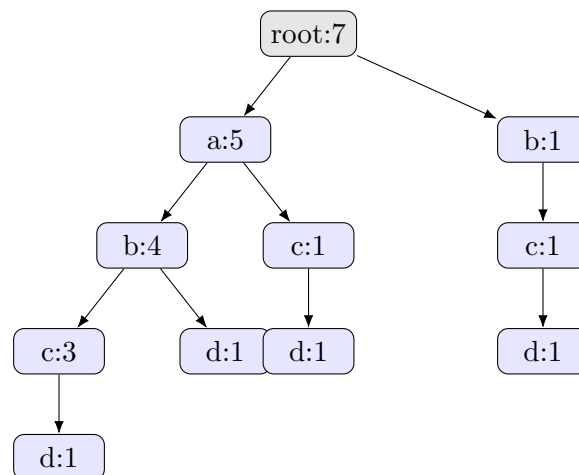


Figure 1.2: FP-tree ví dụ với header: $a:5, b:5, c:5, d:4$ (minsup=3)

Cách xây dựng FP-tree:

Thuật toán xây dựng FP-tree yêu cầu quét cơ sở dữ liệu $\mathcal{D}$ đúng 2 lần:

Lần quét thứ nhất:

1. Quét $\mathcal{D}$ để đếm tần suất xuất hiện (support) của từng item.
2. Loại bỏ các item có support nhỏ hơn ngưỡng *minsup*.
3. Sắp xếp các item phổ biến còn lại theo thứ tự support giảm dần. Gọi danh sách **có thứ tự** này là L .

Lần quét thứ hai:

1. Khởi tạo gốc của cây là một nút **null**.
2. Với mỗi giao dịch (transaction) trong $\mathcal{D}$, loại bỏ các item không phổ biến (không nằm trong danh sách L) và sắp xếp các item còn lại theo đúng thứ tự của L .
3. Tiến hành chèn từng giao dịch đã qua xử lý vào cây như sau:
 - Duyệt giao dịch tuần tự từ trái qua phải, bắt đầu từ nút **gốc** (root) của cây. Giả sử giao dịch hiện tại có dạng: $\{i_1, i_2, \dots, i_k\}$.
 - Kiểm tra xem nút hiện tại đã có nút con trực tiếp nào mang nhãn i_1 hay chưa. Nếu chưa có, ta tạo toàn bộ một nhánh mới xuất phát từ nút hiện tại: $\text{root} \rightarrow i_1 \rightarrow i_2 \rightarrow \dots \rightarrow i_k$, đồng thời thiết lập giá trị **count** của tất cả các nút trên nhánh mới này bằng 1.
 - Nếu đã có sẵn nút con i_1 , con trỏ sẽ đi xuống nút i_1 này và tăng giá trị **count** của nó thêm 1. Sau đó, ta tiếp tục **lặp lại** quy trình trên cho phần còn lại của giao dịch là $\{i_2, \dots, i_k\}$ với vị trí xuất phát mới là nút i_1 .
4. Tồn tại song song là một **Head Table** với nhiệm vụ lưu trữ các node tương ứng cho các item hiện có. Đây là một kỹ thuật về mặt *Tin học* để tối ưu việc tìm kiếm - Không ảnh hưởng đến cấu trúc và tính chất *Toán học* của cây.

Tập cơ sở mẫu điều kiện (Conditional Pattern Base - CPB)

Khái niệm: Tập cơ sở mẫu điều kiện của một mục (hoặc một tập mục) α , ký hiệu là $CPB(\alpha)$, là một cơ sở dữ liệu giao dịch thu nhỏ chứa tất cả các chuỗi tiền tố (*Prefix Paths*) trên cây FP-tree dẫn đến các nút mang nhãn α .

Biểu diễn toán học: Giả sử mục α xuất hiện tại k nhánh khác nhau trên cây FP-tree, tập cơ sở dữ liệu điều kiện được cấu thành bởi:

$$CPB(\alpha) = \{(P_1 : c_1), (P_2 : c_2), \dots, (P_k : c_k)\}$$

Trong đó, P_j đại diện cho chuỗi tiền tố thứ j và c_j là tần suất của nút α trên nhánh cây đó.

Cây FP-tree điều kiện (Conditional FP-tree)

Khái niệm: Cây FP-tree điều kiện của một mục α , ký hiệu là T_α , là một cấu trúc cây FP-tree thu nhỏ được xây dựng trực tiếp từ tập dữ liệu ảo $CPB(\alpha)$ thay vì từ cơ sở dữ liệu gốc ban đầu. Cây này đại diện cho cấu trúc nén thu gọn của tất cả các mặt hàng phổ biến *khi và chỉ khi* chúng xuất hiện đồng thời cùng với tiền tố α .

Mối quan hệ giữa hai cấu trúc: Trong khi $CPB(\alpha)$ đóng vai trò là nguồn dữ liệu thô đầu vào dạng bảng, thì T_α chính là mô hình nén được tối ưu hóa từ nguồn dữ liệu đó để phục vụ cho các bước gọi đệ quy sâu hơn ở các tầng tiếp theo.

Cách khai thác tập phổ biến từ FP-tree

Khai phá tập phổ biến trên FP-tree áp dụng chiến lược chia để trị (divide-and-conquer) dựa trên hậu tố (suffix). Thay vì xét toàn bộ tổ hợp như thuật toán Apriori, thuật toán phân rã không gian tìm kiếm thành các không gian con độc lập.

Như đã đề cập, L là danh sách các item phổ biến được sắp xếp theo support giảm dần. Với mỗi item $\alpha \in L$ (duyệt ngược từ dưới lên, tức là từ item có support thấp nhất đến cao nhất), quy trình đệ quy diễn ra theo 3 bước:

1. Tạo Tập cơ sở kiểu điều kiện (Conditional Pattern Base - CPB):

Tìm tất cả các nhánh trên FP-tree kết thúc bằng nút α . Bỏ qua nút α , phần còn lại của các nhánh này (từ gốc xuống đến trước α) tạo thành CPB của α , ký hiệu là $CPB(\alpha)$. Tần suất (count) của mỗi chuỗi tiền tố trong $CPB(\alpha)$ được gán bằng chính giá trị count của nút α trên nhánh tương ứng.

2. Xây dựng Cây FP-tree điều kiện (Conditional FP-tree):

Từ cơ sở dữ liệu ảo $CPB(\alpha)$, đếm tổng tần suất của các item xuất hiện trong đó. Chỉ giữ lại các item thỏa mãn ngưỡng *minsup* cục bộ để xây dựng nên một cây FP-tree nhỏ hơn dành riêng cho α , ký hiệu là T_α .

3. Khai phá đệ quy:

Nếu T_α không rỗng, thuật toán gọi đệ quy quy trình trên cho T_α . Bất kỳ tập phổ biến β nào sinh ra từ T_α đều được kết hợp với α để tạo thành một tập phổ biến mới: $\beta \cup \{\alpha\}$, với support bằng support của β bên trong T_α .

Chứng minh tính chính xác của quy trình khai thác

Để chứng minh quy trình khai thác từ FP-tree là chính xác, ta chứng minh hai tính chất: Tính đầy đủ (không bỏ sót tập) và Tính chính xác về đếm (tần suất luôn đúng).

A. Chứng minh tính đầy đủ (Completeness - Không bỏ sót tập phổ biến)

Phát biểu: Mọi tập phổ biến X thỏa mãn ngưỡng *minsup* trong cơ sở dữ liệu gốc $\mathcal{D}$ đều được sinh ra từ quy trình FP-Growth.

Chứng minh: Mọi tập phổ biến X đều có thể được biểu diễn dưới dạng $X = Y \cup \{\alpha\}$, trong đó α là item có thứ tự thấp nhất trong X theo danh sách L . Vì thuật toán duyệt từng α

như một hậu tố để lấy ra $CPB(\alpha)$, mọi giao dịch chứa α (và do đó, chắc chắn gồm các giao dịch chứa X) đều được gom đầy đủ vào $CPB(\alpha)$.

Từ định nghĩa có thứ tự của L và cây FP, việc phân rã không gian tìm kiếm theo hậu tố đảm bảo các phân hoạch là rời rạc và vét cạn toàn bộ không gian dữ liệu. Theo tính chất Apriori (Anti-monotonicity), vì X phổ biến, tập con Y cũng phải phổ biến khi đi cùng α . Quá trình xây dựng T_α lọc giữ lại đúng các item đạt *minsup* trong ngữ cảnh của α , do đó tập Y chắc chắn sẽ được tái tạo nguyên vẹn trên cây T_α . Cơ chế đệ quy liên tục phân rã này đảm bảo không một tổ hợp phổ biến nào bị bỏ qua.

B. Chứng minh tính chính xác của tần suất (Count Exactness)

Phát biểu: Tần suất đếm được của tập $X = Y \cup \{\alpha\}$ thông qua cây T_α luôn bằng đúng tần suất xuất hiện thực tế của X trong cơ sở dữ liệu gốc $\mathcal{D}$.

Chứng minh: Trong bước xây dựng FP-tree ban đầu, giá trị *count* của một nút α đại diện cho tổng số lượng giao dịch đi qua đúng nhánh tiền tố đó từ gốc đến α .

Khi tạo $CPB(\alpha)$ ở Bước 1, việc gán tần suất của tiền tố bằng đúng tần suất của nút α phản ánh chính xác số lần các item trong tiền tố đó xuất hiện *đồng thời* với α . Giả sử một item $\gamma \in Y$ xuất hiện trong nhiều nhánh tiền tố khác nhau dẫn đến các nút α ở nhiều vị trí trên cây. Khi xây dựng T_α , tổng tần suất của γ trong cây điều kiện được tính bằng:

$$support(\gamma|\alpha) = \sum_{k=1}^n count(\text{nhánh thứ } k \text{ chứa } \gamma \text{ kết thúc bởi } \alpha)$$

Tổng đại số này chính xác là tổng số lần γ và α xuất hiện cùng nhau trong toàn bộ các giao dịch của $\mathcal{D}$. Tính chất bảo toàn phép cộng này được duy trì ở mọi cấp độ đệ quy cho đến khi hoàn thiện toàn bộ tập Y . Do đó, giá trị đếm cuối cùng của tổ hợp X được thu thập từ các lá của quá trình đệ quy hoàn toàn không bị hao hụt hay dư thừa.

Giải thích về cách Khai thác

Như đã trình bày cách khai thác và chứng minh toán học ở trên, ở đây, chúng ta sẽ làm rõ ý nghĩa của việc sắp xếp L và lý do vì sao lại chọn xây dựng cây tiền tố FP.

Bản chất từ việc sắp xếp L là vô cùng đơn giản và rõ ràng như sau: Việc cố định tất cả các tập phải được sắp xếp lại theo thứ tự trong L sẽ giúp ta xây dựng quy trình đếm như sau: **Với từng item α , ta sẽ đếm toàn bộ các tập phổ biến (có thứ tự) kết thúc tại α .** Ta cần chạy hết toàn bộ *item* trong L là sẽ có ngay toàn bộ tập phổ biến mà không bị sót.

Bản chất của việc xây dựng cây tiền tố cũng là một cách quy ước giúp ta khi truy ngược từ các node của item α về gốc để lấy toàn bộ các tập item xuất hiện cùng với α thu được từ các giao dịch mà không bị đếm sót hay bị thừa. Từ đó giúp xây dựng $CPB(\alpha) \rightarrow T_\alpha$, và **do ta đang đếm các tập phổ biến (có thứ tự) kết thúc tại α** nên việc ta rút các item xuất hiện trong các giao dịch chứa α là đầy đủ.

FP-array

Như đã trình bày ở trên, sau khi ta thu được cây với root, ta cần phải duyệt qua từng item α trong L sau đó thực hiện dựng cây con để in ra các tập phổ biến trong $\mathcal{D}$ có thứ tự kết thúc tại α . Cứ mỗi một α , ta phải duyệt lại cây FP 1 lần để tạo ra cây con T_α - Đây chính là quá trình gây mất thời gian.

Kĩ thuật *FP-array* giúp ta thay vì mất thời gian duyệt lại, ta sẽ xây dựng một mảng gọi là **FP-array** để lưu lại toàn bộ **thông tin về support** cần để xây dựng $T_\alpha \Rightarrow$ Đánh đổi **bộ nhớ** cho **thời gian**.

Cho cây điều kiện T_X với header items $\{i_1, i_2, \dots, i_m\}$ (sắp theo thứ tự giảm dần về support).

FP-array A_X là ma trận $(m-1) \times (m-1)$ trong đó:

$$A_X[j, k] = |\{T \in \text{cond. pattern base of } X : i_k \in T \text{ và } i_j \in T\}|, \quad k < j$$

Cụ thể, $A_X[j, k]$ bằng đúng support của item i_k trong conditional pattern base của $X \cup \{i_j\}$.

	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>
<i>a</i>				
<i>b</i>	4			
<i>c</i>	4	4		
<i>d</i>	3	3	3	

Figure 1.3: FP-array $A_\emptyset$ tương ứng với FP-tree ở Hình 1.2. Hàng *d*: support của *a*, *b*, *c* trong conditional pattern base của *d* đều bằng 3.

Cách sử dụng và Ý nghĩa của FP-array A_X : Khi xây dựng $T_{X \cup \{i_j\}}$ từ T_X , thay vì duyệt toàn bộ linked-list của i_j trong T_X để đếm, ta chỉ cần đọc hàng *j* của A_X :

$$\forall k < j : \text{support của } i_k \text{ trong cond. base của } i_j = A_X[j, k]$$

Kích thước: $|A_X| = \frac{m(m-1)}{2}$. Với $m = 5000$ item, FP-array chiếm khoảng 50 MB – chấp nhận được.

Ngưỡng thưa/dày: Nếu 1/4 trên của cây chứa $< 15\%$ tổng số nút, cây đang dày (dense) – không cần FP-array vì cây nhỏ gọn, lần duyệt đã nhanh. Ngược lại (cây thưa, bushy), FP-array tiết kiệm đáng kể thời gian.

Chúng ta đã nắm được các khái niệm cần thiết, phần tiếp theo ta sẽ phân tích mã giả (pseudocode) của thuật toán FP-growth* từ đó ta sẽ dễ dàng nắm bắt được cách liên kết giữa FP-growth và FP-array trong việc cải thiện tốc độ.

1.2.3 Danh mục ký hiệu và chú thích các biến trong thuật toán

- *T*: Đối tượng cây FP-tree hiện tại đang được xét trong tầng đệ quy.

- $T.base$: Tập tiền tố điều kiện (*conditional prefix*) đã được tích lũy từ các tầng đệ quy trước đó. Đối với cây gốc ban đầu, $T.base = \emptyset$.
- $T.header$: Danh sách các mục (*items*) phổ biến quản lý bởi cây T , được sắp xếp theo thứ tự tần suất giảm dần.
- $T.FP-array$ (A_T): Ma trận tam giác chứa tần suất xuất hiện đồng thời của các cặp mục (*co-occurrence frequent array*) tương ứng với cây T .

1.2.4 Mã giả thuật toán - Pseudocode

Algorithm 1: FPgrowth*(T)

Input: Cây T với thuộc tính: $T.base$, $T.header$, $T.FP-array$ (A_T)

Output: Tập tất cả FI tương ứng với T

```

1 if  $T$  chỉ chứa một nhánh  $B$  then
2   foreach tập con  $Y \subseteq B$  do
3     xuất  $T.base \cup Y$  với support = count nhỏ nhất trong  $Y$ ;
4   return
5 foreach item  $i$  trong  $T.header$  (từ ít phổ biến  $\rightarrow$  nhiều) do
6    $Y \leftarrow T.base \cup \{i\}$ ;
7   xuất  $Y$  với  $i.count$ ;
8   if  $T.FP-array$   $A_T$  đã định nghĩa then
9      $H_Y \leftarrow$  xây header của  $T_Y$  trực tiếp từ hàng  $i$  trong  $A_T$ ;
10    (bỏ qua lần duyệt 1 – cải tiến cốt lõi);
11  else
12     $H_Y \leftarrow$  duyệt  $T$  qua linked-list của  $i$  để lấy các item phổ biến;
13   $T_Y \leftarrow \text{ConstructSubTree}(T, i, H_Y)$ ;
14  //Xây  $T_Y$  bằng cách duyệt linked-list của  $i$  trong  $T$ 
15  //Đồng thời xây  $A_Y$  (nếu thoả điều kiện về độ hiệu quả) (FP-array
    cho  $T_Y$ )
16  if  $T_Y \neq \emptyset$  then
17    gọi FPgrowth*( $T_Y$ );

```

Algorithm 2: ConstructSubTree(T, i, H_Y)

Input: Cây cũ T , mục dữ liệu i , danh sách header mới H_Y đã qua bộ lọc *minsup***Output:** Cây điều kiện mới T_Y có thể kèm theo ma trận tích lũy A_Y

```

1 Khởi tạo một cây rỗng  $T_Y$  với  $T_Y.header \leftarrow H_Y$ ;
2  $T_Y.base \leftarrow T.base \cup \{i\}$ ;
3 // Kiểm tra điều kiện hiệu quả (Độ thưa/dày của cấu trúc cây con)
4 if  $T_Y$  thỏa mãn điều kiện cây thưa (bushy/sparse)  $\wedge |H_Y| > ngưỡng$  then
5     Khởi tạo ma trận  $A_Y$  kích thước  $|H_Y| \times |H_Y|$  toàn ký tự 0;
6      $T_Y.FP-array \leftarrow A_Y$ ;
7 else
8      $T_Y.FP-array \leftarrow Null$ ;
9 // Duyệt qua danh sách liên kết của mục  $i$  trên cây cũ  $T$ 
10 foreach nút  $N$  thuộc danh sách liên kết của  $i$  trong cây  $T$  do
11      $c \leftarrow N.count$ ;
12      $P \leftarrow$  Chuỗi tiền tố điều kiện đi từ nút  $N$  ngược lên Root của cây  $T$ ;
13     Loại bỏ tất cả các mục trong  $P$  không xuất hiện trong danh sách  $H_Y$ ;
14     if  $P \neq \emptyset$  then
15         Chèn chuỗi tiền tố đã lọc  $P$  với tần suất  $c$  vào cây mới  $T_Y$ ;
16         if  $T_Y.FP-array \neq Null$  then
17             // Tính toán song song: Tích lũy tần suất cặp vào ma trận  $A_Y$ 
18             foreach item  $i_j \in P$  do
19                 foreach item  $i_k \in P$  với  $k < j$  do
20                      $A_Y[j, k] += c$ ;
21 return  $T_Y$ ;
```

Algorithm 3: Xây dựng FPTree($\mathcal{D}$, minsup)**Input:** CSDL $\mathcal{D}$, ngưỡng minsup**Output:** FP-tree $T_\emptyset$ kèm FP-array $A_\emptyset$

- 1 **Lần quét 1:** Đếm support mọi item; lọc item có support $\geq \text{minsup}$;
- 2 Sắp xếp item theo support giảm dần $\rightarrow$ thứ tự header;
- 3 Khởi tạo $T_\emptyset$ và $A_\emptyset \leftarrow \mathbf{0}$;
- 4 **Lần quét 2:** **foreach** giao dịch $T \in \mathcal{D}$ **do**
 - 5 Lọc item không phổ biến; sắp T theo thứ tự header;
 - 6 Chèn T vào FP-tree $T_\emptyset$;
 - 7 **foreach** cặp $(i_k, i_j) \in T$ với $k < j$ **do**
 - 8 $A_\emptyset[j, k] += 1$;
- 9 **return** $(T_\emptyset, A_\emptyset)$;

1.2.5 Phân tích độ phức tạp**Độ phức tạp thời gian**

- *Xây dựng FP-tree ban đầu:* $O(\sum_{T \in \mathcal{D}} |T|) = O(|\mathcal{D}| \cdot \bar{l})$ với $\bar{l}$ là độ dài trung bình của một giao dịch.
- *Cập nhật FP-array gốc:* $O\left(\sum_{T \in \mathcal{D}} \frac{l_T(l_T-1)}{2}\right) = O(|\mathcal{D}| \cdot \bar{l}^2)$ với $\bar{l}^2$ là trung bình bình phương độ dài giao dịch sau khi lọc, và l_T là số lượng mục phổ biến (frequent items) giữ lại trong từng giao dịch T .
- **Trường hợp tốt nhất (Cây đơn nhánh – Single-Path):** $O(|\mathcal{D}| \cdot \bar{l}^2 + 2^k)$ với k là số lượng mục dữ liệu phổ biến trên nhánh đơn. Trong đó, chi phí bao gồm giai đoạn khởi tạo cấu trúc gốc $O(|\mathcal{D}| \cdot \bar{l}^2)$ và chi phí khai thác tổ hợp $O(2^k)$. Độ phức tạp $O(k)$ của thao tác kiểm tra điều kiện đơn nhánh là vô cùng nhỏ, bị hấp thụ hoàn toàn bởi hàm mũ $O(2^k)$ khi thuật toán sinh trực tiếp các tập phổ biến (thông qua bộ sinh tổ hợp/bitmask) mà không cần gọi đệ quy sâu.
- **Trường hợp trung bình:** $O(|\mathcal{D}| \cdot \bar{l}^2 + |\text{FI}| \cdot m)$ với $|\text{FI}|$ là tổng số tập phổ biến được xuất ra và m là số lượng mục dữ liệu trong bảng chỉ mục header. Chi phí này phản ánh hai giai đoạn: tiền xử lý $O(|\mathcal{D}| \cdot \bar{l}^2)$ và khai thác đệ quy $O(|\text{FI}| \cdot m)$. Chi phí khai thác ở đây phụ thuộc tuyến tính vào kích thước không gian kết quả (output-sensitive), trong đó thao tác tra cứu tần suất cục bộ để sinh mỗi tập điều kiện được tối ưu hóa bằng mảng FP-array với giới hạn chặn trên chỉ là $O(m)$.
- **Trường hợp xấu nhất:** $O(|\mathcal{D}| \cdot \bar{l}^2 + 2^m)$. Tình huống này xảy ra khi ngưỡng minsup được thiết lập quá thấp trên một cơ sở dữ liệu cực kỳ dày đặc (*Dense Database*). Khi đó, cấu trúc cây không thể phân nhánh thu hẹp và thuật toán bắt buộc phải duyệt qua toàn bộ không gian trạng thái theo hàm mũ để sinh ra đầy đủ $2^m - 1$ tập con. Tuy

nhien, trong khai phá dữ liệu thực nghiệm, tính chất thưa (*Sparsity*) tự nhiên của dữ liệu cùng cơ chế cắt tỉa của ngưỡng minsup luôn giúp bề gãy sớm các nhánh đệ quy, ngăn hiện tượng bùng nổ chi phí này.

Lợi ích cốt lõi của FP-array: Trong thuật toán FP-growth truyền thống, với mỗi cây điều kiện, hệ thống phải duyệt ngược toàn bộ danh sách liên kết (*linked-list*) của mục i với chi phí $O(\sum |P|)$ (với $|P|$ là độ dài các chuỗi tiền tố) chỉ để đếm tần suất và lọc ra các mục phổ biến cục bộ. Nhờ cấu trúc mở rộng FP-array, **FPgrowth*** giảm chi phí này xuống trực tiếp chỉ còn $O(m)$ bằng cách đọc trên hàng tương ứng của ma trận tích lũy. Hơn thế nữa, **FPgrowth*** có tính thích nghi (adaptive) cao: thuật toán tự động ngừng cấp phát FP-array cho các cây con nếu phát hiện cấu trúc quá dày đặc, đảm bảo chi phí cập nhật mảng không bao giờ làm suy giảm hiệu năng tổng thể.

Độ phức tạp không gian

- **FP-tree:** Chặn trên không gian trong trường hợp xấu nhất là $O(|\mathcal{D}| \cdot \bar{l})$, với $\bar{l}$ là chiều dài trung bình của giao dịch. Tuy nhiên, nhờ cơ chế nén bằng cách chia sẻ chung các chuỗi tiền tố từ những giao dịch trùng lặp, kích thước thực tế của FP-tree thường nhỏ hơn rất nhiều so với CSDL gốc $\mathcal{D}$.
- **FP-array:** Cây gốc $T_\emptyset$ cần một ma trận kích thước $m \times m$, tương đương không gian $O(m^2/2)$ với m là số lượng mục phổ biến. Mặc dù thuật toán liên tục sinh ra các mảng điều kiện mới A_Y trong quá trình gọi đệ quy, tổng chi phí không gian tại một thời điểm bất kỳ vẫn bị chặn bởi $O(m^2)$. Điều này được lý giải bởi ba yếu tố:
 1. **Đặc tả duyệt sâu (DFS):** Không gian của cây và mảng điều kiện sẽ được giải phóng ngay lập tức khi thoát khỏi hàm đệ quy. Bộ nhớ chỉ lưu trữ các mảng nằm trên đường dẫn đệ quy hiện tại.
 2. **Kích thước thu hẹp:** Số lượng mục phổ biến cục bộ m_i giảm đi rất nhanh theo độ sâu đệ quy. Tổng dung lượng của các mảng trên một nhánh $\sum O(m_i^2)$ thực chất được hội tụ và áp đảo bởi chi phí $O(m^2)$ của mảng gốc ban đầu.
 3. **Điều kiện khởi tạo:** Thuật toán tối ưu hóa bằng cách không cấp phát mảng A_Y nếu dữ liệu nhánh quá dày (dense) hoặc số lượng mục phổ biến cục bộ nhỏ hơn ngưỡng quy định ($|H_Y| \leq \text{ngưỡng}$).
- **Tổng chi phí bộ nhớ lớn nhất (Peak Memory):** Ước lượng tiệm cận khoảng $O(|T_\emptyset| + m^2/2)$, trong đó $|T_\emptyset|$ là tổng số lượng nút thực tế của cây FP-tree gốc.

Các yếu tố ảnh hưởng hiệu năng

Table 1.1: Ảnh hưởng của các tham số đến hiệu năng của thuật toán FP-growth*

Tham số	Tăng lên	Giảm xuống
minsup	Ít FI hơn, cấu trúc cây thu nhỏ, tốc độ thực thi nhanh hơn	Nhiều FI hơn, cây phình to, tốc độ xử lý chậm hơn
n (số giao dịch)	Cây có xu hướng rộng hơn (<i>sparse</i>), tốn thời gian quét dữ liệu ban đầu	Cấu trúc cây nhỏ gọn, giảm thời gian nạp dữ liệu
$\bar{l}$ (độ dài trung bình)	Cây sâu và rộng hơn, chi phí cập nhật ma trận cấp tăng theo hàm bình phương	Cây nông, kích thước ma trận FP-array nhỏ gọn
m (số item phổ biến)	Không gian mảng $O(m^2)$ tăng nhanh, tăng chi phí duyệt header điều kiện	Mảng FP-array thu hẹp, thuật toán chạy rất nhanh
Độ dày/thưa (<i>Density</i>)	Dense: Khả năng nén của cây cực tốt, nhưng ma trận FP-array ít phát huy tối ưu hiệu năng	Sparse: Cây phân nhánh mạnh, nhưng ma trận FP-array đạt hiệu quả tối ưu vượt trội

1.2.6 So sánh lịch sử

Table 1.2: Dòng thời gian phát triển của các thuật toán FIM

Năm	Thuật toán	Đóng góp	Hạn chế
1993	Apriori	Breadth-first; tính chất anti-monotone để tĩa ứng viên	l lần quét CSDL; sinh ứng viên tốn kém
1994	Apriori (cải tiến)	Hash-tree, heuristics tĩa ứng viên	Vẫn nhiều lần quét CSDL
2000	FP-growth	Nén CSDL vào FP-tree; chỉ 2 lần quét; không sinh ứng viên	2 lần duyệt cây mỗi mức đệ quy
2003	FPmax	Khai thác MFI từ FP-tree; MFI-tree kiểm tra tối đại	Chưa có FP-array
2003–2005	Họ FP-growth*	FP-array: bỏ qua duyệt cây lần 1; xử lý mượt mà dữ liệu thưa thớt	Bộ nhớ $O(m^2)$ cho mảng có thể bùng nổ nếu m lớn
2003+	Eclat, dEclat	Biểu diễn dọc (tidset/diff-set); hiệu quả bộ nhớ	Giao tidset chậm khi CSDL lớn

Họ thuật toán mở rộng bao gồm **FP-growth***, **FPmax*** và **FPclose** đã giải quyết triệt để *nút thắt cổ chai* của FP-growth truyền thống (lần duyệt đầu tiên khi xây dựng cây điều kiện). Các thuật toán này đã được trình diễn tại hội thảo thực nghiệm FIMI Workshop 2003

và chính thức được công bố trên tạp chí IEEE TKDE năm 2005. Trong đó, ban tổ chức FIMI '03 đã chính thức đề cử **FPmax*** và **FPclose** là các thuật toán tốt nhất cho bài toán khai thác tập phổ biến tối đại (MFI) và tập phổ biến đóng (CFI).

Chapter 2

Ví dụ minh họa tay (Step-by-step)

2.1 Ví dụ 1: cơ sở

2.1.1 Cơ sở dữ liệu

Table 2.1: CSDL đồ chơi – 5 giao dịch, 6 item (1–6), minsup = 3

TID	Giao dịch gốc
T_1	1, 2, 3, 5, 6
T_2	1, 2, 4, 5
T_3	1, 4
T_4	2, 4, 6
T_5	1, 2, 3, 5

2.1.2 Xây dựng FP-tree $T_\emptyset$, FP-array $A_\emptyset$

Lần quét 1: Đếm tần suất

Do ngưỡng minsup = 3, các mục 3 và 6 chỉ xuất hiện 2 lần nên bị loại bỏ hoàn toàn khỏi không gian tìm kiếm.

Table 2.2: Tần suất item và kết quả lọc (minsup=3)

Item	Count	Phổ biến?	Thứ tự header
1	4	Có	1
2	4	Có	2
3	2	Không	–
4	3	Có	3
5	3	Có	4
6	2	Không	–

Header table lúc này chỉ còn 4 item: 1:4, 2:4, 4:3, 5:3.

Lần quét 2: Xây dựng FP-tree và FP-array gốc

Mỗi giao dịch được lọc bỏ item không phổ biến (3, 6) và sắp xếp nghiêm ngặt theo thứ tự danh sách header mới:

Table 2.3: Giao dịch sau lọc và sắp xếp theo tần suất giảm dần

TID	Giao dịch gốc	Sau lọc + sắp
T_1	1, 2, 3, 5, 6	[1, 2, 5]
T_2	1, 2, 4, 5	[1, 2, 4, 5]
T_3	1, 4	[1, 4]
T_4	2, 4, 6	[2, 4]
T_5	1, 2, 3, 5	[1, 2, 5]

Ta sẽ nhìn trước kết quả của FP-tree $T_\emptyset$ và FP-array $A_\emptyset$ phải có dạng như sau:

FP-tree kết quả:

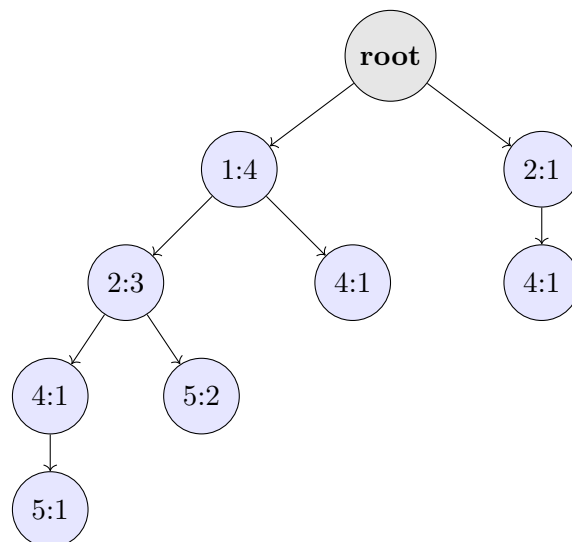


Figure 2.1: FP-tree sau khi chèn 5 giao dịch đã lọc

FP-array $A_\emptyset$:

	1	2	4
2	3		
4	2	2	
5	3	3	1

Figure 2.2: FP-array $A_\emptyset$: $A[j, k]$ tích lũy tần suất đồng xuất hiện. (Ví dụ: $T_2[1, 2, 4, 5]$ cộng dồn cho cặp (5, 4), (5, 2), (5, 1), (4, 2), (4, 1), (2, 1)).

Chi tiết xây dựng $T_\emptyset, A_\emptyset$ theo thuật toán 3

Sau lần duyệt 1, ta đã thu được cơ sở dữ liệu đã được loại bỏ các item *không phổ biến* được sắp lại theo thứ tự *phổ biến nhiều* $\rightarrow$ *phổ biến ít*.

Ta sẽ minh họa bằng cách duyệt qua từng giao dịch trong cơ sở dữ liệu đã lọc (gồm T_1 đến T_5) để cập nhật đồng bộ cây $T_\emptyset$ và ma trận $A_\emptyset$.

1. Khởi tạo $A_\emptyset$ có kích thước 3×3 (tam giác dưới) với toàn bộ giá trị là 0 và $T_\emptyset$ chỉ chứa nút **root**.
2. **Giao dịch** $T_1 = [1, 2, 5]$: Ta chèn chuỗi này vào cây $T_\emptyset$ thành một nhánh mới. Đồng thời cập nhật ma trận $A_\emptyset$ bằng cách cộng 1 vào các ô tương ứng với các cặp $(2, 1)$, $(5, 1)$, $(5, 2)$.

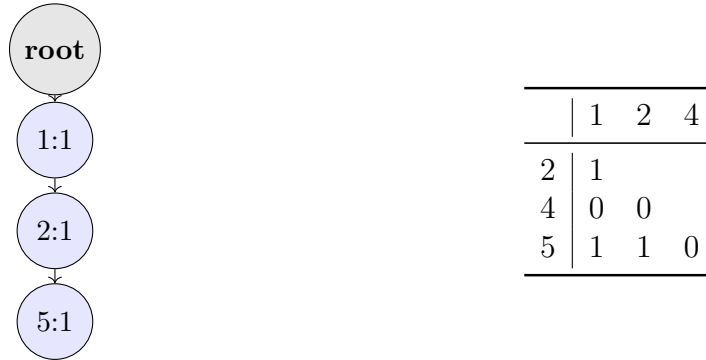


Figure 2.3: Trạng thái $T_\emptyset$ và $A_\emptyset$ sau khi quét $T_1 = [1, 2, 5]$

3. **Giao dịch** $T_2 = [1, 2, 4, 5]$: Cây đi theo nhánh có sẵn $1 \rightarrow 2$ (tăng count lên 2), sau đó chẻ nhánh mới cho $4 \rightarrow 5$. Ma trận cộng dồn các cặp $(2, 1)$, $(4, 1)$, $(4, 2)$, $(5, 1)$, $(5, 2)$, $(5, 4)$.

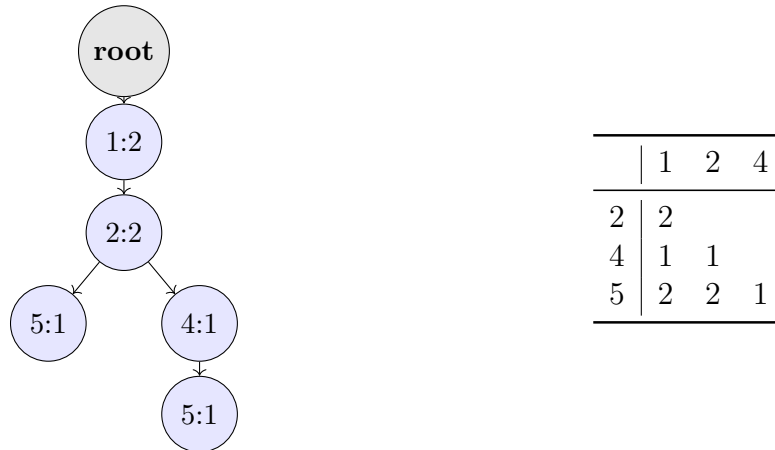
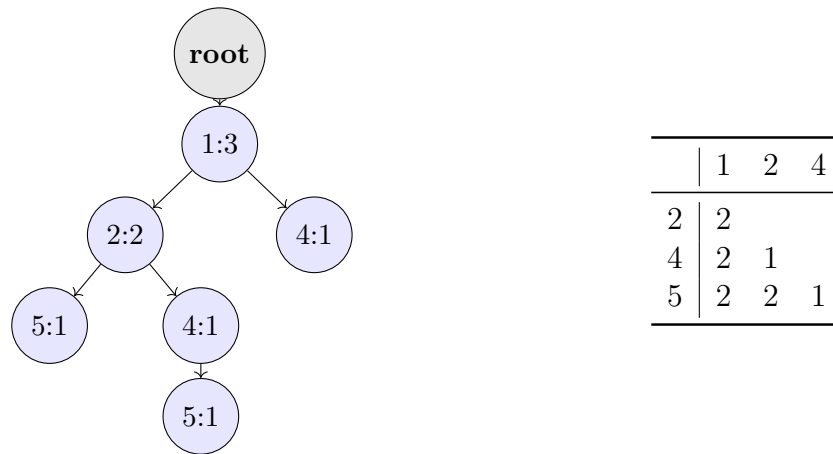
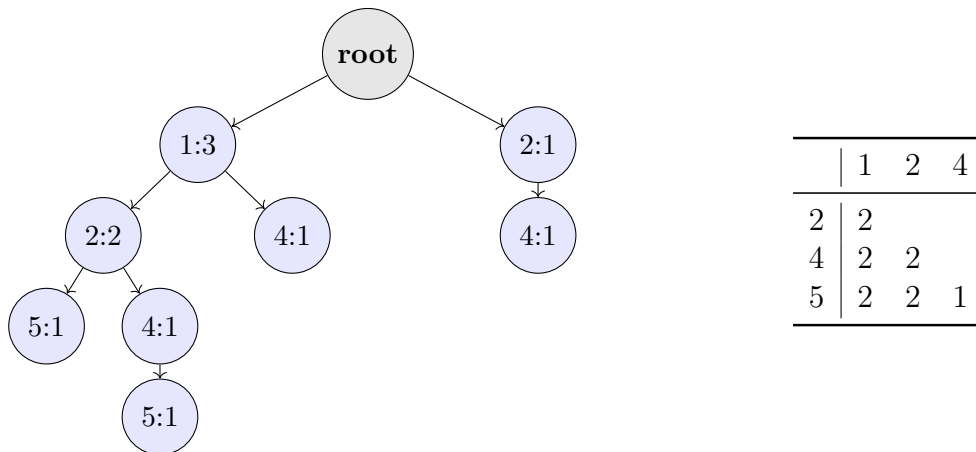


Figure 2.4: Trạng thái $T_\emptyset$ và $A_\emptyset$ sau khi quét $T_2 = [1, 2, 4, 5]$

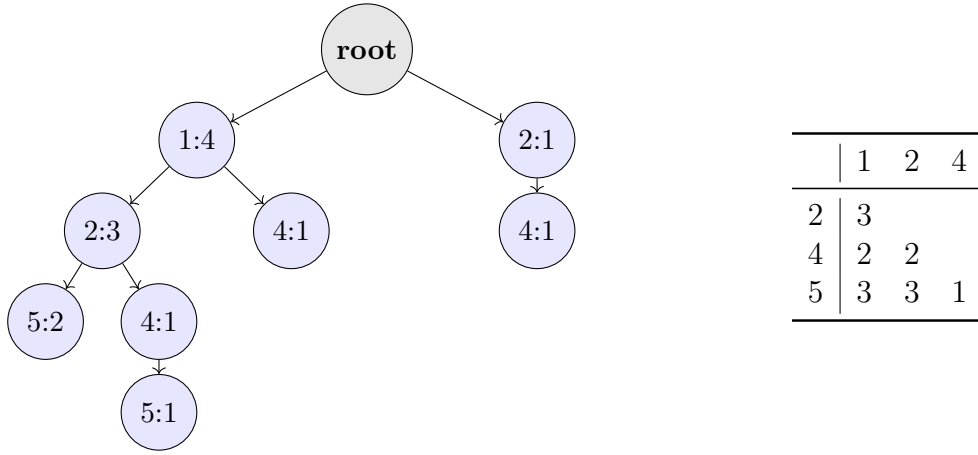
4. **Giao dịch** $T_3 = [1, 4]$: Cây đi qua nút 1 (tăng count lên 3), chẻ nhánh mới cho nút 4. Ma trận chỉ cộng thêm 1 cho cặp $(4, 1)$.

Figure 2.5: Trạng thái $T_\emptyset$ và $A_\emptyset$ sau khi quét $T_3 = [1, 4]$

5. **Giao dịch** $T_4 = [2, 4]$: Giao dịch này không bắt đầu bằng 1, nên tạo một nhánh hoàn toàn mới từ **root**. Ma trận cộng thêm 1 cho cặp $(4, 2)$.

Figure 2.6: Trạng thái $T_\emptyset$ và $A_\emptyset$ sau khi quét $T_4 = [2, 4]$

6. **Giao dịch** $T_5 = [1, 2, 5]$: Quay lại nhánh chính $1 \rightarrow 2 \rightarrow 5$ và cập nhật tần suất (đều tăng thêm 1). Ma trận tiếp tục cộng dồn các cặp $(2, 1)$, $(5, 1)$, $(5, 2)$. Ta thu được trạng thái hoàn chỉnh cuối cùng.

Figure 2.7: Trạng thái HOÀN CHỈNH của T_0 và A_0 sau T_5

2.1.3 Quá trình khai thác tập phổ biến bằng FP-growth* theo thuật toán 1

Với ngưỡng $minsup = 3$, dựa trên kết quả đếm, danh sách các item phổ biến sắp xếp giảm dần là $L = \{1, 2, 4, 5\}$. Thuật toán sẽ duyệt ngược từ dưới lên (từ item có support thấp nhất): xét lần lượt $5 \rightarrow 4 \rightarrow 2 \rightarrow 1$.

1. Khai thác với item 5:

(a) Khởi tạo tập và xuất kết quả cơ sở:

- Đặt $Y \leftarrow T_0.base \cup \{5\} = \{5\}$.
- Xuất $Y = \{5\}$ với tần suất bằng 3 (tổng count của 5 trên cây).

(b) Xây dựng Header H_5 (Bỏ qua duyệt cây nhờ A_T):

- Vì $T_0.FP\text{-array}$ (A_0) đã được định nghĩa, ta tra cứu trực tiếp hàng liên quan đến mục 5 trong ma trận A_0 : $A(5, 1) = 3$, $A(5, 2) = 3$, $A(5, 4) = 1$.
- Giữ lại các mục đạt $\geq minsup$, ta thu được header điều kiện: $H_5 = \{1 : 3, 2 : 3\}$. Mục 4 bị loại bỏ.

(Đây là cải tiến cốt lõi: Thuật toán không cần lần theo linked-list của 5 chỉ để đếm tần suất các item xuất hiện cùng nó như FP-Growth truyền thống).

(c) Gọi hàm ConstructSubTree($T_0, 5, H_5$):

- Khởi tạo cây rỗng T_5 , $T_5.header = \{1, 2\}$, $T_5.base = \{5\}$.
- Giả sử điều kiện tạo FP-array thỏa mãn, khởi tạo A_5 toàn số 0.
- Lần theo danh sách liên kết (linked-list) của mục 5 trên cây cũ T_0 , ta có 2 nút:
 - **Nút thứ 1** ($c = 2$): Tiên tổ ngược lên Root là $P = \{1, 2\}$. Tất cả đều thuộc H_5 . Chèn P vào T_5 với tần suất 2. Cập nhật $A_5(2, 1) += 2$.

- **Nút thứ 2** ($c = 1$): Tiền tố ngược lên Root là $P = \{1, 2, 4\}$. Loại bỏ $4 \notin H_5$, thu được $P' = \{1, 2\}$. Chèn P' vào nhánh có sẵn trên T_5 với tần suất 1. Cập nhật $A_5(2, 1) += 1$.
- **Kết quả:** Ta thu được cây T_5 và ma trận A_5 như hình dưới.



Figure 2.8: Cây điều kiện T_5 và ma trận A_5 trả về từ ConstructSubTree

(d) **Gọi đệ quy FPgrowth*(T_5):**

- Trong ngữ cảnh gọi đệ quy này, $T_5.\text{base} = \{5\}$.
- Thuật toán kiểm tra điều kiện đầu tiên: "*Nếu T chỉ chứa một nhánh $B...$* ". Ta thấy cây T_5 hoàn toàn chỉ là một đường thẳng $B = \{1, 2\}$ với count tương ứng đều là 3.
- Thuật toán kích hoạt nhánh rẽ (single path branch): Sinh mọi tập con $Y \subseteq \{1, 2\}$ và ghép với $T_5.\text{base}$:
 - $Y = \{1\} \Rightarrow$ **Xuất** $\{1\} \cup \{5\} = \{1, 5\}$ với **count** = 3.
 - $Y = \{2\} \Rightarrow$ **Xuất** $\{2\} \cup \{5\} = \{2, 5\}$ với **count** = 3.
 - $Y = \{1, 2\} \Rightarrow$ **Xuất** $\{1, 2\} \cup \{5\} = \{1, 2, 5\}$ với **count** = 3.
- Lệnh **return** được gọi, kết thúc ngay lập tức bước đệ quy của nhánh mục 5 mà không cần tiến hành duyệt đệ quy sâu hơn.

2. Khai thác với item 4:

(a) **Khởi tạo tập và xuất kết quả cơ sở:**

- Đặt $Y \leftarrow T_\emptyset.\text{base} \cup \{4\} = \{4\}$.
- **Xuất** $Y = \{4\}$ với tần suất bằng 3 (tổng count của 4 trên cây gốc).

(b) **Xây dựng Header H_4 :**

- Tra cứu trực tiếp hàng/cột liên quan đến mục 4 trong ma trận $A_\emptyset$: $A(4, 1) = 2$, $A(4, 2) = 2$.
- Vì cả hai giá trị đều $< \text{minsup}$ (ngưỡng là 3), không có mục nào thỏa mãn. Ta thu được header điều kiện rỗng: $H_4 = \emptyset$.

(c) **Gọi hàm ConstructSubTree($T_\emptyset, 4, H_4$):**

- Vì $H_4 = \emptyset$, hàm trả về trực tiếp cây điều kiện rỗng $T_4 = \emptyset$.

(d) **Gọi đệ quy $\text{FPgrowth}^*(T_4)$:**

- Vì $T_4 = \emptyset$, thuật toán không thỏa mãn điều kiện gọi đệ quy. Kết thúc ngay nhánh khai thác của mục 4.

3. **Khai thác với item 2:**(a) **Khởi tạo tập và xuất kết quả cơ sở:**

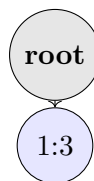
- Đặt $Y \leftarrow T_\emptyset.\text{base} \cup \{2\} = \{2\}$.
- **Xuất** $Y = \{2\}$ với tần suất bằng 4 (tổng nhánh $1 \rightarrow 2$ là 3, nhánh trực tiếp từ root là 1).

(b) **Xây dựng Header H_2 :**

- Tra cứu ma trận $A_\emptyset$ cho mục 2 (chỉ xét các mục đứng trước nó): $A(2, 1) = 3$.
- Giá trị này $\geq \text{minsup}$, do đó ta thu được header điều kiện: $H_2 = \{1 : 3\}$.

(c) **Gọi hàm $\text{ConstructSubTree}(T_\emptyset, 2, H_2)$:**

- Khởi tạo cây rỗng T_2 , $T_2.\text{header} = \{1\}$, $T_2.\text{base} = \{2\}$.
- Do $|H_2| = 1$, cấu trúc quá nhỏ để xây dựng ma trận FP-array có ý nghĩa, ta gán $T_2.\text{FP-array} = \text{Null}$.
- Lần theo **linked-list** của mục 2 trên $T_\emptyset$, ta xét 2 nút:
 - **Nút thứ 1** ($c = 3$): Nằm nhánh dưới nút 1. Tiền tố ngược lên Root là $P = \{1\}$. Thỏa mãn H_2 . Chèn P vào T_2 với tần suất 3.
 - **Nút thứ 2** ($c = 1$): Nối trực tiếp từ Root. Tiền tố $P = \emptyset$. Không có item nào được chèn.
- **Kết quả:** Cây T_2 thu được chỉ có một nhánh duy nhất.

Figure 2.9: Cây điều kiện T_2 trả về từ ConstructSubTree (không tạo ma trận A_2)(d) **Gọi đệ quy $\text{FPgrowth}^*(T_2)$:**

- Ngữ cảnh hiện tại: $T_2.\text{base} = \{2\}$.
- Cây T_2 chỉ chứa một nhánh thẳng $B = \{1\}$ với $\text{count} = 3$. Rơi vào điều kiện cắt tỉa nhánh đơn.
- Sinh mọi tập con $Y \subseteq \{1\}$:
 - $Y = \{1\} \Rightarrow$ **Xuất** $\{1\} \cup \{2\} = \{1, 2\}$ với $\text{count} = 3$.
- Lệnh **return** được gọi, kết thúc đệ quy nhánh mục 2.

4. Khai thác với item 1:

(a) Khởi tạo tập và xuất kết quả cơ sở:

- Đặt $Y \leftarrow T_{\emptyset}.\text{base} \cup \{1\} = \{1\}$.
- Xuất $Y = \{1\}$ với tần suất bằng 4 (tổng count của 1 trên cây gốc).

(b) Xây dựng Header H_1 và dùng phân nhánh:

- Mục 1 là phần tử đứng đầu tiên (phổ biến nhất) trong danh sách L đã sắp xếp. Không có bất kỳ mục nào đứng trước nó để có thể làm tiền tố.
- Do đó, $H_1 = \emptyset$ và $T_1 = \emptyset$.
- Không có tập phổ biến nào lớn hơn chứa mục 1 đóng vai trò làm hậu tố cuối cùng. Kết thúc hoàn toàn thuật toán FP-growth*.

Kết luận: Qua phương pháp FP-growth* kết hợp ma trận, ta nhanh chóng khai thác được toàn bộ các tập phổ biến (ngoài các tập 1-itemset cơ sở) mà giảm thiểu tối đa việc duyệt và tái tạo cây. Danh sách tập phổ biến thu được là: $\{1, 5\} : 3, \{2, 5\} : 3, \{1, 2\} : 3, \{1, 2, 5\} : 3$. **Kết quả thực nghiệm đầu ra khớp hoàn toàn 100% với thuật toán brute-force độc lập, chứng minh tính logic của FP-array chạy trong mã giả.**

2.2 Ví dụ 2: Tình huống đặc biệt

2.2.1 Mô tả tình huống đặc biệt

Khi tất cả giao dịch chia sẻ chung một chuỗi tiền tố lồng nhau theo thứ tự, cấu trúc cây FP-tree kết quả **chỉ chứa một nhánh duy nhất đi từ gốc đến lá**. Đây là tình huống đặc biệt quan trọng vì:

- FPgrowth* nhận diện ngay lập tức thông qua hàm kiểm tra điều kiện dừng và thoát khỏi cấu trúc đệ quy sâu.
- Mọi tập con không rỗng được sinh ra từ các mục trên đường dẫn này đều chắc chắn là tập mục phổ biến.
- Độ hỗ trợ (*support*) của một tập con Y bất kỳ chính bằng giá trị tần suất nhỏ nhất của các mục nằm trong tập con đó: $\text{support} = \min_{x \in Y} \{x.\text{count}\}$.
- Thuật toán hoàn toàn không cần cấp phát bộ nhớ để xây dựng bất kỳ cây điều kiện (*conditional FP-tree*) hay ma trận mảng phụ nào khác.

2.2.2 Cơ sở dữ liệu

Đặt ngưỡng $\text{minsup} = 1$. Kết quả tính toán tần suất thu được: 1:5, 2:4, 3:3, 4:2, 5:1. Tất cả các mục đều thỏa mãn điều kiện phổ biến (≥ 1).

Table 2.4: CSDL tình huống đặc biệt – các giao dịch lồng nhau tạo tiền tố chung

TID	Giao dịch
T_1	1, 2, 3, 4, 5
T_2	1, 2, 3, 4
T_3	1, 2, 3
T_4	1, 2
T_5	1

2.2.3 Xây dựng FP-tree

Do cấu trúc dữ liệu đầu vào có tính chất tập con lồng nhau hoàn hảo ($T_5 \subset T_4 \subset T_3 \subset T_2 \subset T_1$), quá trình chèn các chuỗi sau khi sắp xếp vào cây sẽ không sinh ra bất kỳ nhánh rẽ nào.

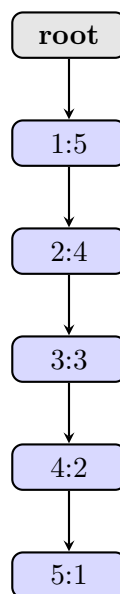


Figure 2.10: FP-tree tạo thành một nhánh đơn duy nhất: $\text{root} \rightarrow 1:5 \rightarrow 2:4 \rightarrow 3:3 \rightarrow 4:2 \rightarrow 5:1$

2.2.4 Xử lý trong FPgrowth*

Tại tầng gọi hàm đầu tiên, điều kiện kiểm tra cấu trúc cây xác định cây chỉ chứa một nhánh duy nhất $B = [(1, 5), (2, 4), (3, 3), (4, 2), (5, 1)]$.

Thuật toán lập tức bỏ qua quy trình duyệt danh sách liên kết truyền thống, tiến hành áp dụng công thức tổ hợp lũy thừa để liệt kê tất cả $2^5 - 1 = 31$ tập mục phổ biến không rỗng từ tập nền tương ứng:

2.2.5 Ý nghĩa kỹ thuật của trường hợp tối ưu này

1. **Tối ưu hóa hiệu năng vượt trội:** Thay vì phải thực hiện 5 tầng đệ quy lồng nhau phức tạp và liên tục cấp phát/giải phóng bộ nhớ cho các cấu trúc dữ liệu trung gian,

Table 2.5: Danh sách tất cả 31 tập phổ biến được sinh trực tiếp từ cây đơn nhánh

STT	Itemset	Support (min)	STT	Itemset	Support (min)
1	{1}	5	17	{1, 2, 4}	$\min(5, 4, 2) = 2$
2	{2}	4	18	{1, 2, 5}	$\min(5, 4, 1) = 1$
3	{3}	3	19	{1, 3, 4}	$\min(5, 3, 2) = 2$
4	{4}	2	20	{1, 3, 5}	$\min(5, 3, 1) = 1$
5	{5}	1	21	{1, 4, 5}	$\min(5, 2, 1) = 1$
6	{1, 2}	$\min(5, 4) = 4$	22	{2, 3, 4}	$\min(4, 3, 2) = 2$
7	{1, 3}	$\min(5, 3) = 3$	23	{2, 3, 5}	$\min(4, 3, 1) = 1$
8	{1, 4}	$\min(5, 2) = 2$	24	{2, 4, 5}	$\min(4, 2, 1) = 1$
9	{1, 5}	$\min(5, 1) = 1$	25	{3, 4, 5}	$\min(3, 2, 1) = 1$
10	{2, 3}	$\min(4, 3) = 3$	26	{1, 2, 3, 4}	$\min(5, 4, 3, 2) = 2$
11	{2, 4}	$\min(4, 2) = 2$	27	{1, 2, 3, 5}	$\min(5, 4, 3, 1) = 1$
12	{2, 5}	$\min(4, 1) = 1$	28	{1, 2, 4, 5}	$\min(5, 4, 2, 1) = 1$
13	{3, 4}	$\min(3, 2) = 2$	29	{1, 3, 4, 5}	$\min(5, 3, 2, 1) = 1$
14	{3, 5}	$\min(3, 1) = 1$	30	{2, 3, 4, 5}	$\min(4, 3, 2, 1) = 1$
15	{4, 5}	$\min(2, 1) = 1$	31	{1, 2, 3, 4, 5}	$\min(5, 4, 3, 2, 1) = 1$
16	{1, 2, 3}	$\min(5, 4, 3) = 3$			

FPgrowth* chỉ sử dụng một bộ sinh chuỗi nhị phân (*bitmask generator*) để đẩy trực tiếp dữ liệu ra tập kết quả với độ phức tạp thời gian đạt mức tối ưu cục bộ.

- Bảo toàn tính đúng đắn toán học:** Do tất cả các mục trên nhánh đơn này được sắp xếp theo mối quan hệ cha-con tuyến tính nối tiếp nhau, bất kỳ giao dịch nào chứa mục ở sâu phía dưới (ví dụ mục 5) chắc chắn phải chứa toàn bộ các mục tổ tiên đứng trước nó (1, 2, 3, 4). Do đó, việc lấy giá trị đếm nhỏ nhất (*minimum count*) làm độ hỗ trợ cho tập mục phối hợp là hoàn toàn chính xác theo định nghĩa lý thuyết tập hợp.
- Tần suất xuất hiện cao:** Trong thực tế khai phá dữ liệu lớn, mặc dù cây FP-tree gốc ban đầu rất lớn và phân nhánh phức tạp (*bushy tree*), nhưng sau khi trải qua nhiều tầng chia cắt đệ quy theo điều kiện tiền tố, các cây con điều kiện ở các tầng sâu rất nhanh chóng hội tụ về dạng đơn nhánh này. Bước chặn này đóng vai trò quyết định giúp thuật toán bẻ gãy nhánh đệ quy sớm, ngăn ngừa hiện tượng bùng nổ thời gian tính toán.

Chapter 3

Cài đặt

3.1 Môi trường và công cụ

- **Ngôn ngữ:** Julia ≥ 1.9 (tận dụng multiple dispatch, `struct` kiểu cụ thể, `@inbounds`, `@elapsed`).
- **Thư viện:** không sử dụng thư viện FIM bên ngoài, sử dụng thư viện chuẩn `Printf`, `Random`, `Test`.
- **Công cụ tham chiếu:** SPMF (Java, cần JDK ≥ 21) để đối chiếu tính đúng đắn và hiệu năng.
- **Hệ điều hành:** Windows, Linux/macOS.
- **Mã nguồn:** <https://github.com/Tranchande/Data-mining-lab02/tree/main>

3.2 Kiến trúc mã nguồn

Toàn bộ cài đặt được gói trong module `FPGrowthStar` , nạp lần lượt các tệp con theo đúng thứ tự phụ thuộc.

Listing 3.1: Cấu trúc thư mục dự án

```
1 itemset/
2 |-- README.md
3 |-- Project.toml                # Moi truong Julia
4 |-- src/
5 |   |-- FPGrowthStar.jl
6 |   |-- structures.jl
7 |   |-- utils.jl
8 |   |-- rules.jl
9 |   |-- main.jl
10 |   |-- algorithm/
11 |       |-- fpgrowth_star.jl
12 |-- test/
13 |   |-- runtests.jl
14 |   |-- test_correctness.jl
15 |   |-- test_benchmark.jl
16 |   |-- test_rules.jl
17 |   |-- spmf.jl
18 |-- data/
19 |-- results/
20 |-- SPMF/spmf.jar
21 |-- notebooks/demo.ipynb
22 |-- docs/
```

3.3 Yêu cầu cài đặt

3.3.1 Cài đặt cơ bản

Thuật toán được cài lại từ đầu theo bài báo gốc (from scratch) gồm bốn thành phần chính:

1. **Xây dựng FP-tree:** hai lần quét CSDL – lần 1 đếm tần số item, lần 2 lọc item không phổ biến, sắp mỗi giao dịch theo thứ tự support giảm dần rồi chèn vào cây (chia sẻ tiền tố). Mỗi item được nối vào danh sách liên kết trong bảng đầu.
2. **Khai thác đệ quy:** xét item theo thứ tự support *tăng dần*, sinh tập phổ biến và đệ quy trên cây điều kiện.

3. **Kỹ thuật mảng đếm:** cốt lõi của FP-Growth*, xác định item phổ biến của CSDL điều kiện trong *một* lần duyệt (xem Mục 3.3.2).
4. **Tối ưu nhánh đơn:** khi cây là một nhánh, liệt kê trực tiếp $2^k - 1$ tập con bằng bitmask.

Kết quả là toàn bộ tập phổ biến cùng support. Tính đúng đắn được kiểm chứng bằng bộ unit test tự động `test/runtests.jl` (đối chiếu thuật toán vét cạn trên ≥ 5 CSDL nhỏ) và đối chiếu trực tiếp với SPMF.

3.3.2 Tối ưu hóa bộ nhớ và tốc độ

Các kỹ thuật tối ưu đã cài đặt:

1. **Loại bỏ bước kiểm tra thích nghi và cấp phát mảng đếm vô điều kiện (Điểm cải tiến cốt lõi):** Trong bài báo gốc của Grahne & Zhu, tác giả đề xuất một cơ chế thích nghi (adaptive): chỉ cấp phát FP-array khi cây con đạt độ thưa nhất định ($|H_Y| > \text{ngưỡng}$), nếu cây quá dày thì bỏ qua để tránh lãng phí bộ nhớ $O(m^2)$. Tuy nhiên, việc duyệt cây để kiểm tra mật độ ở mỗi tầng đệ quy gây ra chi phí quản lý (overhead) rất lớn. Nhóm đã tối ưu bằng cách **loại bỏ hoàn toàn bước kiểm tra điều kiện này**. Thuật toán tiến hành khởi tạo vô điều kiện một mảng đếm 1D `count_arr` có kích thước $O(\text{max_item})$ ngay khi xét một mục dữ liệu α . Giải pháp này triệt tiêu hoàn toàn chi phí kiểm tra điều kiện của bài báo gốc, đơn giản hóa luồng thực thi mà vẫn tối ưu RAM nhờ tận dụng cơ chế giải phóng bộ nhớ tự động theo phạm vi (scope) đệ quy của Julia.
2. **Truy xuất siêu tốc $O(1)$ kết hợp @inbounds:** Tận dụng lợi thế của bộ nhớ mảng tĩnh 1 chiều, việc tra cứu và cộng dồn tần suất tổ tiên từ chuỗi liên kết nút được thực hiện với độ phức tạp thời gian hằng số. Nhóm áp dụng thêm macro `@inbounds` trong các vòng lặp đếm nóng (hot loops) nhằm vô hiệu hóa cơ chế kiểm tra biên (bounds checking) của hệ thống, tối đa hóa tốc độ thực thi phần cứng.
3. **Nén FP-tree và Đảm bảo tính tất định (Determinism):** Nhóm không chỉ sắp xếp item theo *support* giảm dần để nén chia sẻ tiền tố, mà còn chủ động chèn thêm quy tắc tie-break dựa trên ID item (thông qua hàm `by = i -> (-frequent[i], i)`). Điều này đảm bảo tính tái lập (reproducibility) tuyệt đối, hình dáng cây FP-tree được xây dựng luôn cố định qua mọi lần chạy độc lập.
4. **Chèn Node-link $O(1)$:** Khác với cách cài đặt node-link thông thường, cấu trúc `HeaderEntry` của nhóm được thiết kế lưu trữ đồng thời hai con trỏ `head` và `tail`. Khi chèn một nút mới, thay vì phải duyệt tuần tự danh sách mất $O(N)$, thuật toán móc nối trực tiếp nút vào phần đuôi với chi phí hằng số $O(1)$.

5. **Tối ưu nhánh đơn và Phòng vệ lập trình (Defensive Programming):** Khi nhận diện cây điều kiện là một nhánh đơn tuyến tính, thuật toán lập tức áp dụng toán tử bitmask để sinh trực tiếp $2^k - 1$ tập con, loại bỏ hoàn toàn các tầng đệ quy xây cây rỗng tốn kém. Nổi bật hơn so với lý thuyết gốc, nhóm đã thiết lập thêm chốt chặn $depth > 60$ nhằm ngăn ngừa triệt để lỗi tràn số nguyên (Integer Overflow) trên hệ thống 64-bit khi tiến hành dịch bit.

Đo lường: Cài đặt của nhóm được đối chiếu trực tiếp với phần mềm tham chiếu chuẩn SPMF. Thực nghiệm cho thấy, nhờ việc loại bỏ overhead kiểm tra điều kiện kết hợp với kỹ thuật mảng 1D cấp thấp, FP-Growth* của nhóm vượt qua SPMF về tốc độ thực thi ở mọi mức *minsup*, đồng thời thể hiện khả năng kiểm soát bộ nhớ xuất sắc trên các tập dữ liệu thưa.

3.3.3 Xử lý đầu vào/đầu ra

Hỗ trợ định dạng SPMF (mỗi dòng một giao dịch, item cách nhau bởi khoảng trắng). Hàm `read_spmf` đọc, `write_results` ghi theo dạng 1 2 3 #SUP: k.

Listing 3.2: Sử dụng qua CLI

```
1 # Cu phap: julia src/main.jl <input_file> <minsup> [output_file]
2 #minsup La so nguyen duong hoac so thuc trong khoang (0,1]
3 #vi du minh hoa
4 julia src/main.jl data/toy/example1.txt 3
5 julia src/main.jl data/benchmark/chess.txt 0.8 results/chess_out.txt
```

Chapter 4

Thực nghiệm và đánh giá

4.1 Tập dữ liệu benchmark

Thực nghiệm trên 4 tập benchmark chuẩn của cộng đồng FIM.

Dataset	#Giao dịch	#Item	AvgLen	Đặc điểm
Chess	3.196	75	37,0	Dày đặc, itemset dài
Mushroom	8.416	119	23,0	Dày đặc, nhiều item
Retail	88.162	~16.470	10,3	Thưa, dữ liệu bán lẻ thực
Accidents	340.183	468	33,8	Rất lớn, dày đặc

Table 4.1: Bốn tập benchmark sử dụng trong thực nghiệm.

4.2 Kết quả thực nghiệm

4.2.1 Kiểm tra tính đúng đắn (Correctness)

So sánh số lượng tập phổ biến và *support* của từng tập giữa cài đặt của nhóm và công cụ tham chiếu SPMF (với CSDL nhỏ, đối chiếu thêm thuật toán vét cạn). Kết quả **khớp 100%** – không có tập nào thiếu/thừa, support trùng khớp.

Dataset	<i>minsup</i>	#FI (nhóm)	#FI (tham chiếu)	Tỉ lệ khớp
example1	3	10	10	100%
example2	1	31	31	100%
Chess	80%	8.227	8.227	100%
Mushroom	30%	2.587	2.587	100%
Retail	1%	159	159	100%
Accidents	80%	149	149	100%

Table 4.2: Đối chiếu tính đúng đắn(toy: vét cạn; benchmark: SPMF).

4.2.2 Thời gian chạy theo *minsup*

(b) Thời gian theo *minsup*: FP-Growth* vs SPMF

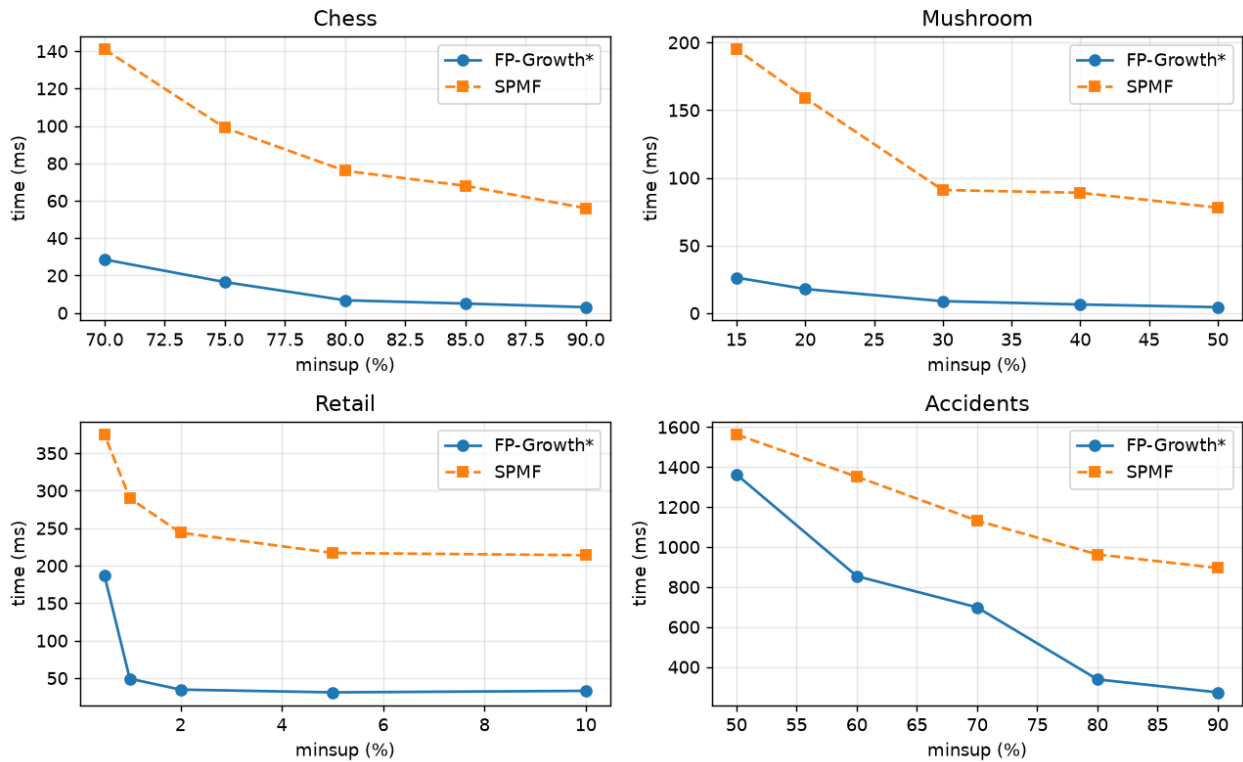


Figure 4.1: Thời gian theo *minsup* của FP-Growth* vs SPMF.

Nhận xét: cài đặt của nhóm nhanh hơn SPMF trên cả 4 CSDL ở mọi mức khảo sát (ví dụ Chess 70%: 28,6 ms so với 141 ms; Accidents 50%: 1.363 ms so với 1.563 ms). Khi *minsup* giảm, số tập phổ biến tăng nhanh nên thời gian cả hai đều tăng. Chênh lệch phản ánh cả yếu tố thuật toán (mảng đếm của FP-Growth* so với FP-Growth) lẫn yếu tố ngôn ngữ/cài đặt (Julia vs Java).

4.2.3 Số lượng FI theo *minsup*

(c) Số frequent itemset theo *minsup*

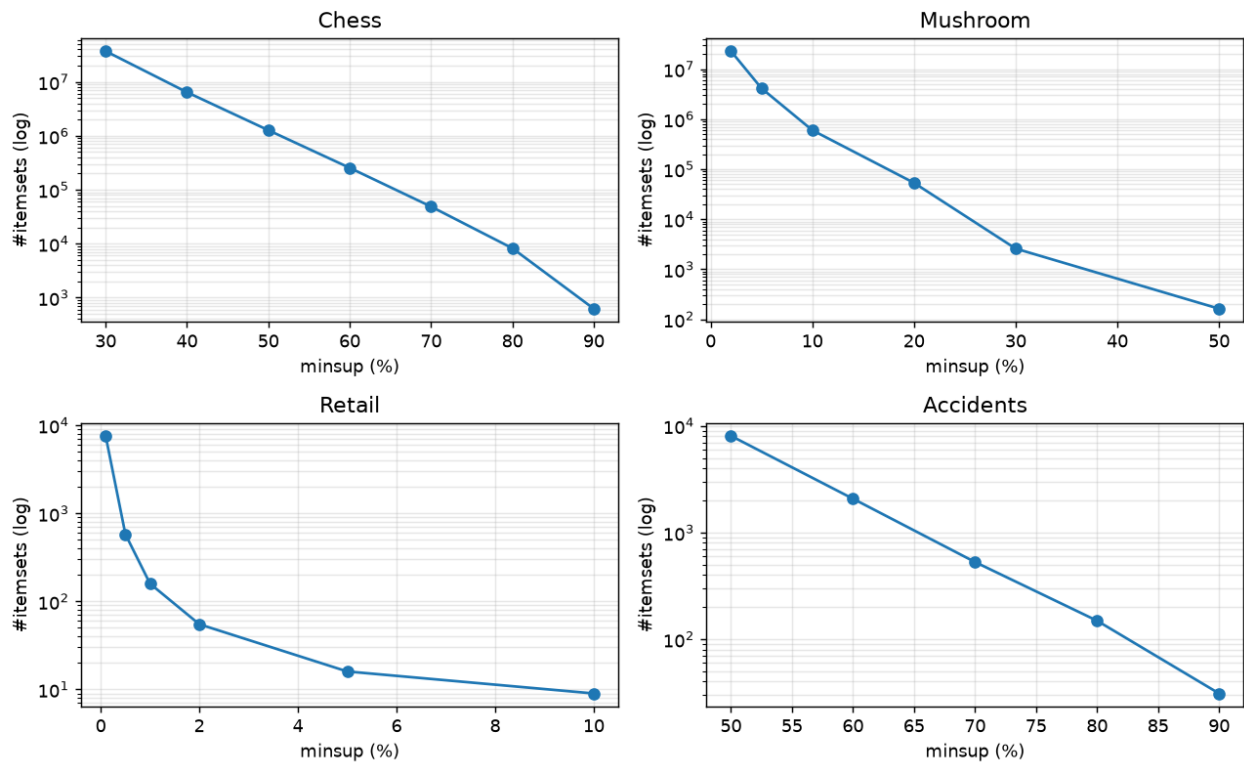


Figure 4.2: Số tập phổ biến theo *minsup*.

Nhận xét: số tập phổ biến tăng theo cấp số nhân khi *minsup* giảm, mạnh nhất ở các CSDL dày (Mushroom ở 15% đã sinh ~ 99.000 tập). CSDL thưa (Retail) tăng chậm hơn nhiều. Đây là nguyên nhân thời gian và bộ nhớ bùng nổ ở *minsup* thấp.

4.2.4 Bộ nhớ sử dụng

Dataset	<i>minsup</i>	FP-Growth* net (MB)	SPMF (MB)
Chess	80%	21,8	19,9
Mushroom	30%	20,5	27,8
Retail	1%	53,1	132,5
Accidents	70%	54,8	87,4

Table 4.3: Bộ nhớ khai phá so với SPMF.

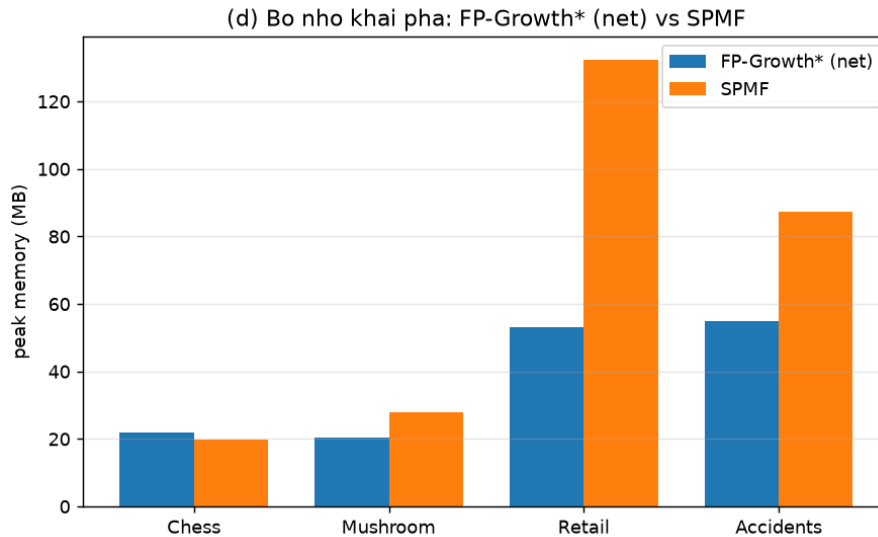


Figure 4.3: Bộ nhớ khai phá: FP-Growth* với SPMF.

Nhận xét: bộ nhớ khai phá thực sự của FP-Growth* *ngang hoặc thấp hơn* SPMF (Retail: 53MB so với 132MB). Tuy nhiên bộ nhớ *tổng* cao do giữ toàn bộ CSDL dưới dạng `Vector{Vector{Int}}` (riêng dữ liệu Accidents đã ~547 MB) – là điểm có thể tối ưu (Mục 4.3.3).

4.2.5 Khả năng mở rộng (Scalability)

Lấy các tập con 10%, 25%, 50%, 75%, 100% giao dịch (Retail ở *minsup* = 1%, Mushroom ở 10%).

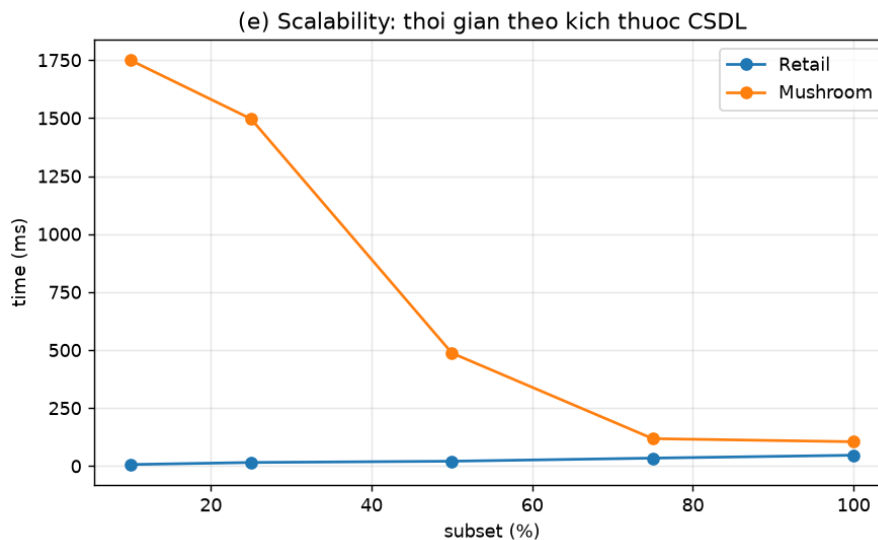


Figure 4.4: Thời gian theo kích thước CSDL.

Nhận xét: trên Retail (thưa), thời gian tăng *gần tuyến tính* theo số giao dịch – thuật toán mở rộng tốt. Trên Mushroom, do *minsup tương đối cố định*, khi tập con nhỏ thì ngưỡng tuyệt đối nhỏ $\Rightarrow$ sinh rất nhiều tập (subset 10% sinh ~4,9 triệu tập), khiến đường cong *giảm* khi CSDL lớn dần – minh họa ảnh hưởng áp đảo của *minsup* so với kích thước.

4.2.6 Ảnh hưởng của độ dài giao dịch trung bình

Sinh CSDL tổng hợp (cố định seed) với $n = 2000$ giao dịch, $|I| = 20$ item, $minsup = 15\%$, tăng dần độ dài giao dịch.

(f) Ảnh hưởng do dài giao dịch

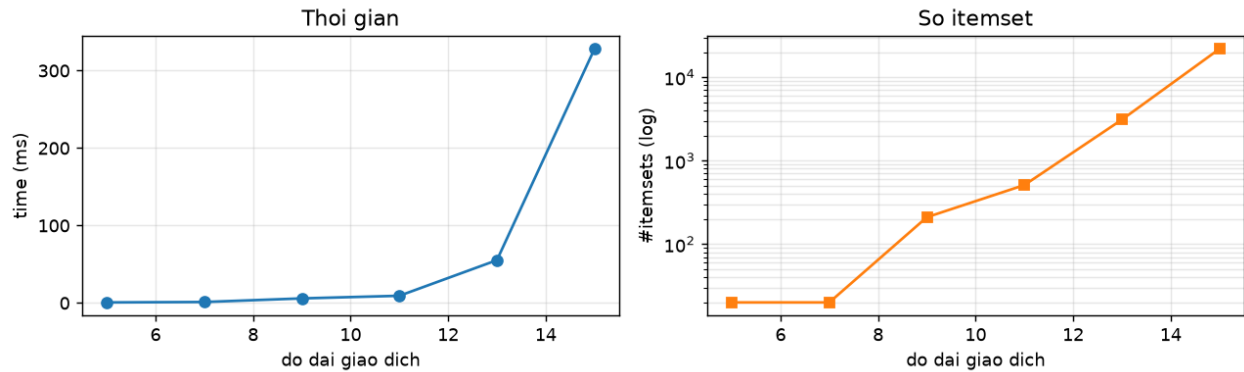


Figure 4.5: Ảnh hưởng độ dài giao dịch

Độ dài	5	7	9	11	13	15
#FI	20	20	210	510	3.103	21.837
Time (ms)	0,7	1,3	5,9	9,3	55,2	328,0

Table 4.4: Số tập phổ biến và thời gian tăng vọt theo độ dài giao dịch.

Nhận xét: giao dịch càng dài thì FP-tree càng rậm, số tổ hợp item cùng xuất hiện càng lớn $\Rightarrow$ số tập phổ biến và thời gian tăng *theo cấp số nhân*. Thuật toán *bị ảnh hưởng mạnh* bởi độ dài giao dịch trung bình.

4.3 Phân tích kết quả

4.3.1 Điểm mạnh

- Kết quả đúng 100% so với SPMF trên mọi CSDL.
- *Nhanh hơn* cài đặt tham chiếu SPMF ở mọi mức $minsup$ nhờ kỹ thuật mảng đếm (chỉ 1 lần duyệt thay vì 2).
- Bộ nhớ khai phá thực sự (net) cạnh tranh, thấp hơn SPMF trên dữ liệu thưa.
- Mở rộng gần tuyến tính theo số giao dịch trên dữ liệu thưa.

4.3.2 Điểm yếu

- Bộ nhớ *tổng* cao do biểu diễn CSDL bằng `Vector{Vector{Int}}` (mỗi giao dịch là một mảng heap riêng, dùng `Int64`).
- Chưa song song hóa – mới khai thác tuần tự trên một luồng.
- Bùng nổ thời gian/bộ nhớ ở *minsup* rất nhỏ trên CSDL dày (bản chất bài toán FIM, không tránh được).

4.3.3 Đề xuất tối ưu hóa tiếp theo

1. **Giảm bộ nhớ biểu diễn:** thay `Vector{Vector{Int}}` bằng `Int32` và/hoặc một mảng phẳng kiểu CSR (giá trị + chỉ số đầu dòng) – ước tính giảm $\geq 50\%$ bộ nhớ giữ CSDL, đặc biệt với Accidents.
2. **Song song hóa:** các nhánh đệ quy ứng với từng item ở mức gốc độc lập nhau, có thể chạy đa luồng bằng `Threads.@spawn` để tận dụng CPU đa nhân.

Chapter 5

Ứng dụng thực tế

5.1 Phân tích giỏ hàng (Market Basket Analysis)

5.1.1 Dữ liệu

Sử dụng tập **Retail** (88.162 giao dịch, nguồn SPMF) – dữ liệu bán lẻ thực từ một cửa hàng tạp hóa tại Bỉ. Mỗi giao dịch là một giỏ hàng; item là ID sản phẩm (ẩn danh).

5.1.2 Phương pháp

Từ tập phổ biến khai thác bởi FP-Growth*, hàm `association_rules` (tệp `src/rules.jl`) của nhóm sinh luật kết hợp $X \Rightarrow Y$ với $X \cap Y = \emptyset$ và $X \cup Y$ là tập phổ biến. Với mỗi luật:

$$\sigma(X \Rightarrow Y) = \sigma(X \cup Y), \quad \text{conf}(X \Rightarrow Y) = \frac{\sigma(X \cup Y)}{\sigma(X)},$$

$$\text{lift}(X \Rightarrow Y) = \frac{\text{conf}(X \Rightarrow Y)}{\sigma(Y)/n} = \frac{n \cdot \sigma(X \cup Y)}{\sigma(X) \sigma(Y)}.$$

Chỉ giữ luật có $\text{conf} \geq \text{minconf}$. Support của mọi tập con X, Y đều có sẵn trong tập phổ biến nhờ tính chất Apriori, nên việc sinh luật không cần quét lại CSDL.

5.1.3 Kết quả

Thiết lập: $\text{minsup} = 0,1\%$ (≥ 89 giao dịch), $\text{minconf} = 50\%$. Thu được **7.589** tập phổ biến $\rightarrow$ **6.192** luật kết hợp. Bảng dưới liệt kê các luật có lift cao nhất.

Tiền đề X (mua)	Kết quả Y (kèm)	Support	Conf	Lift
{1819, 1820}	{796, 3312}	96	0,66	397,07
{796, 3312}	{1819, 1820}	96	0,65	397,07
{1819, 3312}	{796, 1820}	96	0,69	362,00
{697}	{700}	91	0,58	338,34
{3403}	{3536}	125	0,71	305,20
{796, 1819, 3312}	{1820}	96	0,91	318,11
{796, 3312}	{1820}	124	0,84	296,29

Table 5.1: Top luật kết hợp trên Retail (sắp theo lift giảm dần).

5.1.4 Ý nghĩa kinh doanh

Các luật có lift $\gg 1$ (ở đây tới ≈ 397) cho thấy các nhóm sản phẩm được mua cùng nhau vượt xa xác suất ngẫu nhiên – là cơ sở để:

- Sắp xếp hàng hóa cạnh nhau, bán chéo (cross-selling).
- Thiết kế khuyến mãi gói (bundle promotions).
- Gợi ý sản phẩm tại quầy thanh toán: ví dụ luật $\{796, 1819, 3312\} \Rightarrow \{1820\}$ có confidence 0,91 – khi khách đã mua ba món đầu thì gần như chắc chắn mua kèm món 1820.

Lưu ý: item Retail là ID ẩn danh nên diễn giải theo quan hệ thống kê; với CSDL có tên sản phẩm (vd *Groceries*) chỉ cần thay tệp dữ liệu là diễn giải được trực tiếp mà không đổi mã nguồn.

Bibliography

- [1] G. Grahne and J. Zhu, “Fast Algorithms for Frequent Itemset Mining Using FP-Trees,” *IEEE Transactions on Knowledge and Data Engineering*, vol. 17, no. 10, pp. 1347–1362, October 2005.
- [2] J. Han, J. Pei, and Y. Yin, “Mining Frequent Patterns Without Candidate Generation,” *Proc. ACM-SIGMOD International Conference on Management of Data*, pp. 1–12, May 2000.
- [3] R. Agrawal, T. Imielinski, and A. Swami, “Mining Association Rules Between Sets of Items in Large Databases,” *Proc. ACM-SIGMOD International Conference on Management of Data*, pp. 207–216, May 1993.
- [4] R. Agrawal and R. Srikant, “Fast Algorithms for Mining Association Rules,” *Proc. International Conference on Very Large Data Bases*, pp. 487–499, September 1994.
- [5] M. J. Zaki, “Scalable Algorithms for Association Mining,” *IEEE Transactions on Knowledge and Data Engineering*, vol. 12, no. 3, pp. 372–390, 2000.
- [6] J. Wang, J. Han, and J. Pei, “CLOSET+: Searching for the Best Strategies for Mining Frequent Closed Itemsets,” *Proc. ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 236–245, August 2003.